Cahier des charges

Plateforme intelligente de gestion des réclamations clients (architecture microservices et classification par IA)

Projet de Fin d'Année (PFA) 2026-2027 · Sujet 20 · EMSI Casablanca · Filière IIR / DSIC

Réalisé par :El Bachyr Maraim / Mekouar Karim · Encadrante : Mme Faris Sophia

1. Contexte et problématique

Les entreprises reçoivent chaque jour des réclamations de clients par des canaux variés (e-mail, téléphone, formulaires). Ces réclamations sont souvent traitées manuellement : elles sont lues une à une, classées à la main puis transmises au bon service. Ce processus est lent, source d'erreurs d'orientation et difficile à suivre, ce qui dégrade la satisfaction des clients.

Le projet consiste à concevoir une plateforme web qui centralise les réclamations, automatise leur classification grâce à un modèle de Machine Learning et facilite leur suivi par les agents et les responsables. La plateforme repose sur une architecture microservices afin de séparer clairement les fonctionnalités métier et d'intégrer le modèle d'IA comme un service indépendant.

Problématique : comment automatiser la catégorisation et l'orientation des réclamations clients tout en gardant un contrôle humain, au sein d'une architecture distribuée évolutive ?

2. Objectifs du projet

- Permettre aux clients de soumettre et de suivre leurs réclamations en ligne.
- Mettre en place une architecture microservices avec des services indépendants et communicants.
- Gérer l'authentification et les rôles des utilisateurs (client, agent, administrateur).
- Permettre la consultation, l'affectation et le suivi des réclamations.
- Développer un module d'IA qui classifie automatiquement une réclamation et propose le service chargé de son traitement.
- Permettre aux agents de confirmer ou de corriger les classifications proposées.
- Fournir un tableau de bord de statistiques sur les réclamations.
- Tester la communication entre les microservices et évaluer les performances du modèle d'IA.

3. Périmètre du projet

Inclus : soumission et suivi des réclamations, gestion des rôles, classification IA, affectation, historique, notifications, tableau de bord, évaluation du modèle.

Exclus (hors périmètre de cette version) : paiement, intégration avec un CRM externe, canaux vocaux ou téléphoniques, application mobile native, support multilingue complet (la classification est d'abord développée pour le français).

4. Acteurs du système

Acteur	Rôle
Client	Soumet ses réclamations, suit leur statut, reçoit des notifications
Agent	Traite les réclamations de son service, change leur statut, confirme ou corrige la classification IA
Administrateur	Gère les utilisateurs, les catégories, les services et les affectations, consulte les statistiques
Module IA (acteur système)	Analyse le texte d'une réclamation et propose une catégorie et un service

5. Besoins fonctionnels

Priorités : **H** = haute, **M** = moyenne, **B** = basse.

5.1 Gestion des comptes et des rôles

ID	Besoin	Priorité
BF1	Inscription d'un client avec validation des données	H
BF2	Authentification sécurisée (connexion, déconnexion, jetons)	H
BF3	Gestion des rôles : client, agent, administrateur	H
BF4	Consultation et modification du profil	M
BF5	Réinitialisation du mot de passe	B

5.2 Gestion des réclamations (client)

ID	Besoin	Priorité
BF6	Soumettre une réclamation (titre, description, pièce jointe facultative)	H
BF7	Consulter la liste de ses réclamations et leur statut	H
BF8	Consulter le détail et l'historique d'une réclamation	H
BF9	Ajouter un complément d'information à une réclamation ouverte	M
BF10	Recevoir une notification à chaque changement de statut	M

5.3 Traitement des réclamations (agent)

ID	Besoin	Priorité
BF11	Consulter les réclamations affectées à son service	H
BF12	Changer le statut : nouvelle, en cours, résolue, clôturée	H
BF13	Ajouter des commentaires internes et des réponses au client	H
BF14	Confirmer ou corriger la catégorie et le service proposés par l'IA	H
BF15	Réaffecter une réclamation à un autre service ou agent	M

5.4 Module d'intelligence artificielle

ID	Besoin	Priorité
BF16	Classifier automatiquement le texte d'une réclamation dans une catégorie (ex. Facturation, Support technique, Livraison)	H
BF17	Proposer le service de traitement correspondant à la catégorie	H

ID	Besoin	Priorité
BF18	Afficher un score de confiance avec la prédiction	M
BF19	Basculer vers un classement manuel si la confiance est trop faible ou si le service IA est indisponible	H
BF20	Enregistrer les corrections des agents afin de pouvoir réentraîner le modèle	M

5.5 Administration et statistiques

ID	Besoin	Priorité
BF21	Gérer les utilisateurs (création, désactivation, attribution des rôles)	H
BF22	Gérer les catégories et les services de traitement	H
BF23	Affecter ou réaffecter manuellement des réclamations	M
BF24	Tableau de bord : nombre de réclamations par catégorie, par statut et par service	H
BF25	Indicateurs : délai moyen de traitement, taux de résolution, taux de correction des prédictions IA	M
BF26	Exporter les statistiques (CSV ou PDF)	B

6. Besoins non fonctionnels

Catégorie	Exigence
Sécurité	Authentification par jetons JWT (ou OAuth2), mots de passe hachés, contrôle d'accès par rôle sur chaque service, communication en HTTPS, protection contre les attaques courantes (injection, XSS, CSRF)
Performance	Classification d'une réclamation en moins de 2 secondes ; temps de réponse des API inférieur à 1 seconde en usage normal

Catégorie	Exigence
Disponibilité et résilience	La panne du service IA ne doit pas empêcher la soumission d'une réclamation (délais d'attente, repli vers un classement manuel)
Scalabilité	Services indépendants, déployables et dimensionnables séparément
Maintenabilité	Code modulaire et testé, API documentées avec OpenAPI/Swagger, configuration centralisée
Traçabilité	Historique des statuts et des actions, journaux (logs) centralisés
Ergonomie	Interface simple, claire et responsive
Qualité du modèle IA	Objectif d'au moins 85 % d'accuracy et de F1-score macro sur le jeu de test (valeur cible à confirmer après les premiers essais)
Portabilité	Conteneurisation avec Docker et orchestration locale avec Docker Compose
Protection des données	Conservation limitée aux données nécessaires au traitement, accès restreint aux réclamations selon le rôle

7. Architecture microservices proposée

La plateforme est découpée en services indépendants, chacun avec sa propre base de données.

Service	Responsabilité
API Gateway	Point d'entrée unique, routage, vérification des jetons
Discovery Service (Eureka)	Enregistrement et découverte des services
Config Server	Configuration centralisée
auth-service	Comptes, authentification, rôles
claim-service	Cycle de vie des réclamations, commentaires, historique, affectations

Service	Responsabilité
ai-classifier-service	Classification du texte (API REST en Python, par exemple FastAPI)
notification-service	Notifications par e-mail ou dans l'application
stats-service	Agrégation des données et indicateurs du tableau de bord
Frontend web	Interface utilisateur (React)

Communication : appels synchrones REST (OpenFeign) entre claim-service et ai-classifier-service, avec délai d'attente et repli ; événements asynchrones (par exemple RabbitMQ ou Kafka) pour les notifications et les statistiques.

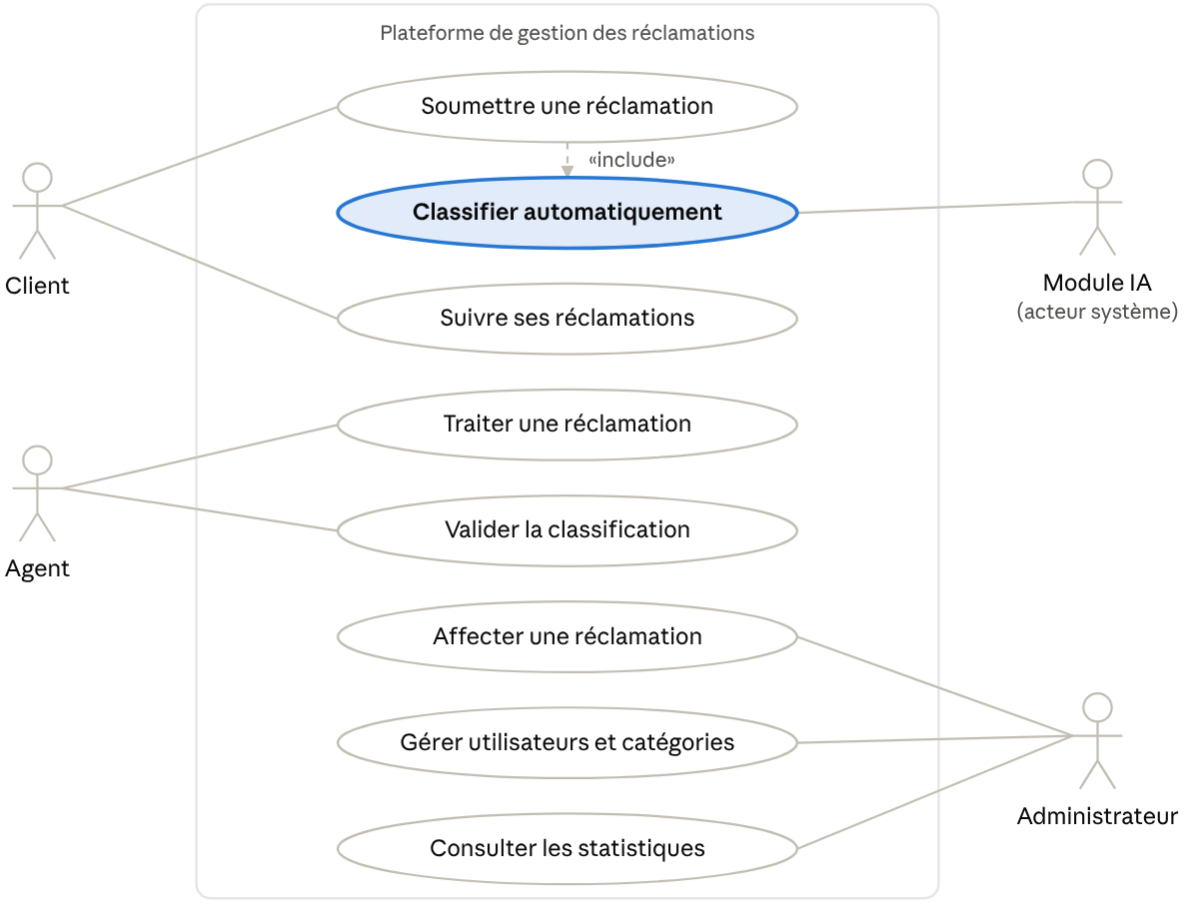
8. Modélisation UML

Les diagrammes suivants accompagnent ce cahier des charges.

8.1 Diagramme de cas d'utilisation

Acteurs : client, agent, administrateur, module IA. Cas d'utilisation principaux : s'authentifier, soumettre une réclamation (qui inclut « classifier automatiquement »), suivre une réclamation, traiter une réclamation, confirmer ou corriger la classification, affecter une réclamation, gérer les utilisateurs et les catégories, consulter les statistiques.

Le client soumet, l'IA classe, l'agent valide, l'administrateur pilote



Tous les acteurs humains s'authentifient avant d'accéder à ces cas d'utilisation.

cas d'utilisation · 4 acteurs, 8 cas

Soumettre une réclamation inclut toujours la classification automatique : le module IA n'intervient que par ce cas.

8.2 Diagramme de classes

Classes principales : User, Role, Claim, Category, Service, Comment, StatusHistory, Attachment, Notification, Prediction (catégorie prédite, score de confiance, correction éventuelle).

La réclamation est au centre : tout le reste s'y rattache

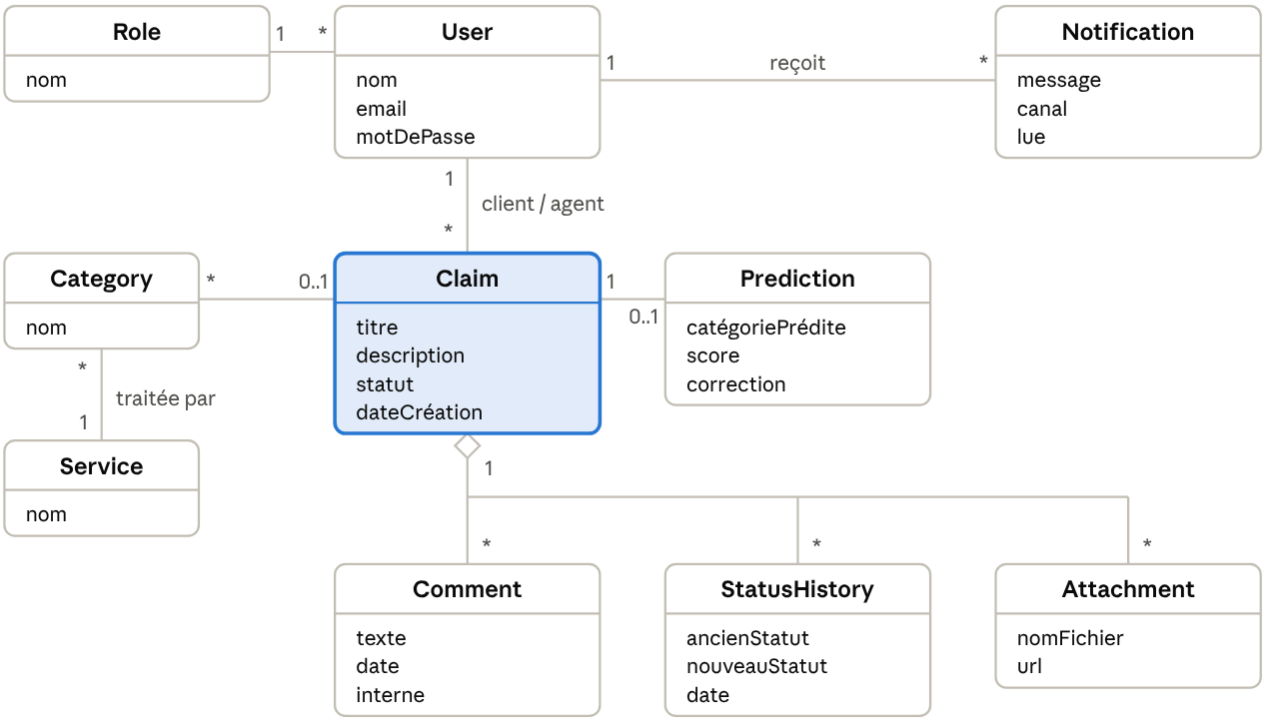


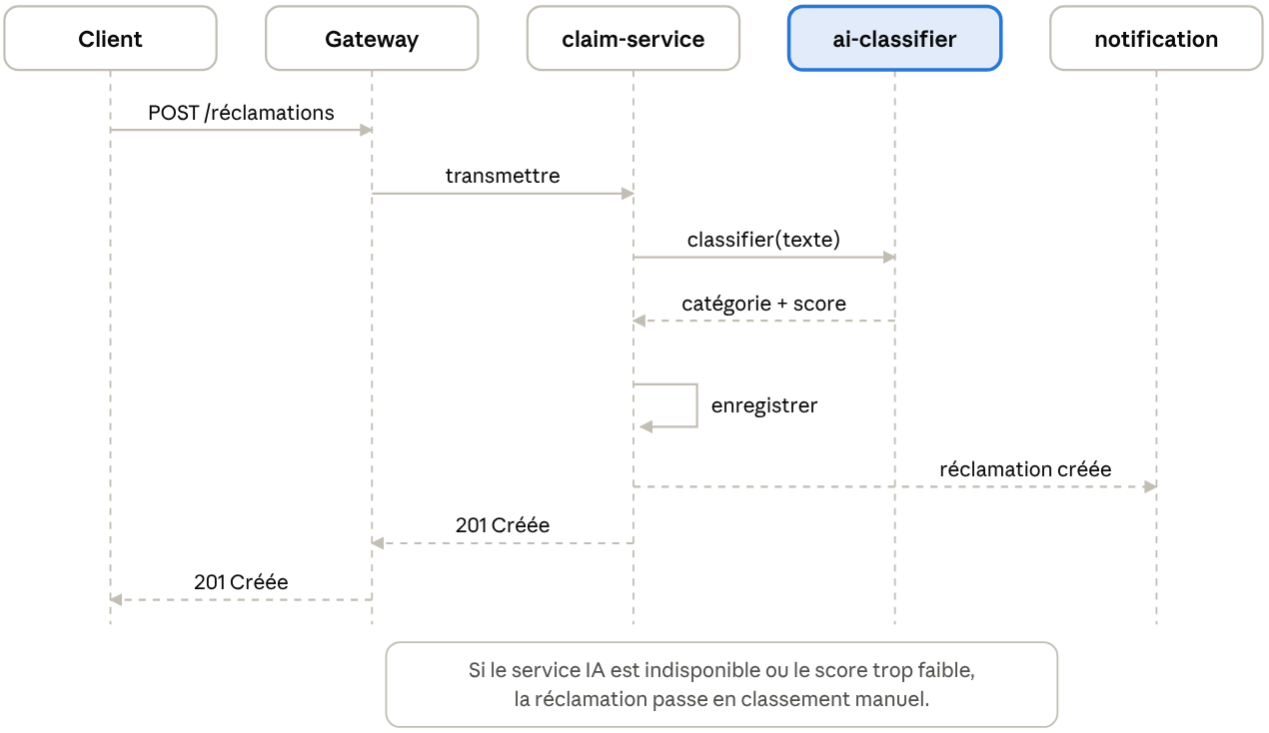
diagramme de classes · 10 classes

Une réclamation appartient à un client, porte au plus une prédiction IA et regroupe ses commentaires, son historique et ses pièces jointes.

8.3 Diagramme de séquence : soumission d'une réclamation

Client → Gateway → claim-service → ai-classifier-service (classification) → enregistrement de la réclamation et de la prédiction → notification-service → réponse au client. Cas alternatif : service IA indisponible ou confiance faible, la réclamation passe en classement manuel.

La classification IA s'insère dans la soumission, avec repli manuel



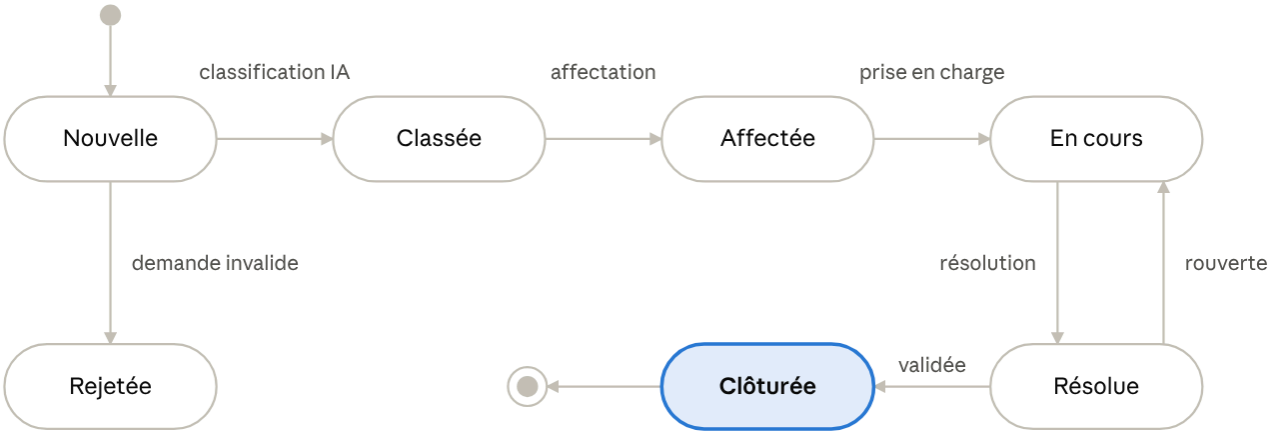
séquence de soumission · 5 participants

La réclamation est enregistrée même si l'IA est indisponible : elle passe alors en classement manuel.

8.4 Diagramme d'états : cycle de vie d'une réclamation

États : nouvelle, classée, affectée, en cours, résolue, clôturée, rejetée. Transitions déclenchées par la classification, l'affectation, le traitement par l'agent et la validation du client.

Une réclamation avance en six états, de Nouvelle à Clôturée



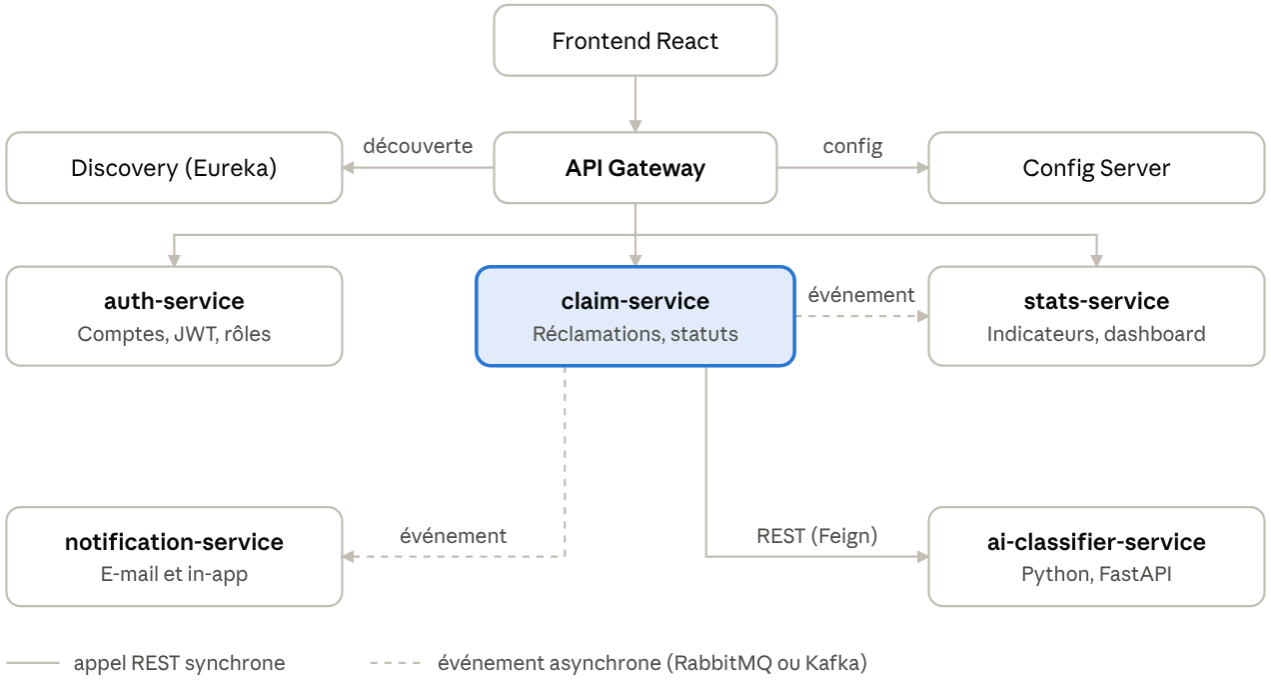
cycle de vie d'une réclamation · 7 états

Une réclamation résolue peut être rouverte par le client ; une demande invalide est rejetée dès sa réception.

8.5 Diagramme de composants et de déploiement

Services, bases de données, passerelle, bus de messages et conteneurs Docker, avec leurs interactions.

Une passerelle unique devant cinq services, chacun avec sa base



Chaque service possède sa propre base de données ; seule la passerelle est exposée.

architecture microservices · 9 composants

Le client n'atteint que la passerelle ; claim-service appelle le service IA en REST et publie des événements pour les notifications et les statistiques.

9. Technologies envisagées

Domaine	Choix proposé
Backend microservices	Spring Boot, Spring Cloud (Gateway, Eureka, Config, OpenFeign)
Service IA	Python, scikit-learn (TF-IDF + régression logistique ou SVM), possibilité d'un modèle préentraîné de type transformeur, FastAPI
Frontend	React

Domaine	Choix proposé
Bases de données	PostgreSQL ou MySQL pour les données métier, une base par service
Messagerie	RabbitMQ ou Kafka
Sécurité	Spring Security, JWT
DevOps	Docker, Docker Compose, Git/GitHub, intégration continue
Documentation et tests	Swagger/OpenAPI, JUnit, Postman

10. Jeu de données et évaluation du modèle IA

- Constitution d'un jeu de données de réclamations textuelles étiquetées (jeu public adapté, complété par des exemples générés ou rédigés par l'équipe).
- Prétraitement : nettoyage, normalisation, vectorisation.
- Comparaison d'au moins deux modèles de classification.
- Métriques : accuracy, précision, rappel, F1-score, matrice de confusion.
- Analyse des erreurs et des limites du modèle.

11. Livrables

- Cahier des charges et dossier de conception UML.
- Code source des microservices et du frontend sur GitHub.
- Modèle d'IA et notebook d'évaluation.
- Application déployable avec Docker Compose.
- Rapport final et démonstration.

12. Planning prévisionnel

Phase	Contenu
1. Analyse et conception	Cahier des charges, UML, choix techniques
2. Socle technique	Gateway, Eureka, Config Server, auth-service
3. Services métier	claim-service, notification-service, stats-service

Phase	Contenu
4. Module IA	Jeu de données, entraînement, ai-classifier-service, intégration
5. Frontend	Interfaces client, agent, administrateur, tableau de bord
6. Tests et évaluation	Tests d'intégration, charge, évaluation du modèle
7. Finalisation	Documentation, rapport, soutenance

13. Risques et mesures

Risque	Mesure
Manque de données étiquetées	Combiner un jeu public et des exemples rédigés, élargir progressivement
Qualité insuffisante du modèle	Comparer plusieurs approches, ajuster le prétraitement, garder la validation humaine
Complexité de l'architecture distribuée	Construire par étapes, tester chaque service, utiliser Docker Compose
Panne du service IA	Délais d'attente, repli vers le classement manuel